Zhuang Liu
SID: 25727277

CS 234 Advanced Networks
Programming Assignment 2

1 Realizing IP Forwarding in P4

1.1 Header Implementation

By observing the header format, we can implement the header as follows:

```
header ipv4_t {
    bit<4>    version;
    bit<4>    ihl;
    bit<8>    diffserv;
    bit<16>   totalLen;
    bit<16>   identification;
    bit<3>    flags;
    bit<13>   fragOffset;
    bit<8>    ttl;
    bit<8>    protocol;
    bit<16>   hdrChecksum;
    ip4Addr_t srcAddr;
    ip4Addr_t dstAddr;
}
```

1.2 Parser Implementation

In the parser, there are three states. The `start` state is the initial state of the parser.

```
state start {
    transition parse_ethernet;
}
```

Then we transit to parse the ethernet header. If there is an IPv4 header, we transit to the IPv4 parsing state. Otherwise, we use an `accept` code to end the parser.

```
state parse_ethernet {
    packet.extract(hdr.ethernet);
    transition select(hdr.ethernet.etherType) {
        TYPE_IPV4: parse_ipv4;
        default: accept;
    }
}
```

The IPv4 header parsing state can be implemented as follows:

```
state parse_ipv4 {
    packet.extract(hdr.ipv4);
    transition accept;
}
```

1.3 Match and Action Table Implementation

To forward the IPv4 table, we need to set the egress port to the assigned port, and set the source and destination address properly. So we can implement the IPv4 forwarding action as follows:

```
action ipv4_forward(macAddr_t dstAddr, egressSpec_t port) {
    standard_metadata.egress_spec = port;
    hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
    hdr.ethernet.dstAddr = dstAddr;
    hdr.ipv4.ttl = hdr.ipv4.ttl - 1;
}
```

Also, we can implement the `drop` action as follows:

```
action drop() {
    mark_to_drop();
}
```

With these two actions, we can utilize them to implement our forwarding table. In the forwarding table, we need the key of Longest Prefix Match(lpm) matches with the longest subnet mask to enter the tunnel. So we can implement the IPv4 forwarding table as follows:

```
    table ipv4_lpm {
        key = {
            hdr.ipv4.dstAddr: lpm;
        }
        actions = {
            ipv4_forward;
            drop;
            NoAction;
        }
        size = 1024;
        default_action = drop();
    }
```

1.4 Control Implementation

If the IPv4 header is valid, we can apply the packet to the control flow:

```
    apply {
        if (hdr.ipv4.isValid()) {
            ipv4_lpm.apply();
        }
    }
```

1.5 Topology Design

Here, we keep the default network topology, which is:

```
    switches 3
    hosts 6
    h1 s1
    s1 h2
    s1 s2
    s2 s3
    s1 s3
    h3 s2
    s2 h4
    h5 s3
    s3 h6
```

Accordingly, the commands will be like this, which uses the command design in `command0.txt` file. Also, the MAC address of the hosts should be configured correspondingly.

```
table_add ipv4_lpm ipv4_forward 10.0.0.1/32 => 0x00000001 1
table_add ipv4_lpm ipv4_forward 10.0.0.2/32 => 0x00000002 2
table_add ipv4_lpm ipv4_forward 10.0.0.3/32 => 0x00000003 3
table_add ipv4_lpm ipv4_forward 10.0.0.4/32 => 0x00000004 3
table_add ipv4_lpm ipv4_forward 10.0.0.5/32 => 0x00000005 4
table_add ipv4_lpm ipv4_forward 10.0.0.6/32 => 0x00000006 4
```

1.6 Verifications

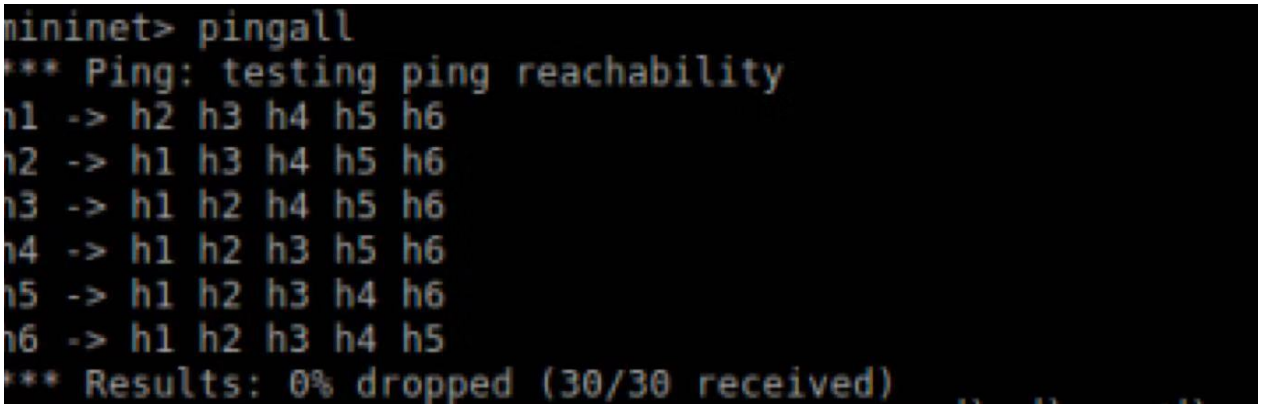
After implemented the `task1.p4` file, we can test our code. At first, we can go to the `mininet` by running shell script:

```
bash run_demo.sh
```

In the `mininet`, we can run the following command to test if all the hosts are connected smoothly:

```
pingall
```

The `ping` result capture is showed as follows:



It is showed that all the packets are sent and received successfully.

2 Implementing IP Tunnel App on ONOS

2.1 P4 Implementation

2.1.1 Header Implementation

By observing the format of the header, we can see that it has two parts, which are Ethernet Header and IPv4. In the `task1.p4` file, `ethernet_t`, `packet_out_header_t` and `packet_in_header_t` are already defined as follows:

```
struct headers_t{
    ethernet_t ethernet;
    packet_out_header_t packet_out;
    packet_in_header_t packet_in;
}
```

So now we only need to implement the tunneling protocol called `my_tunnel_t` and `ipv4_t` protocol in the header. We can add them into the header structure at first:

```
struct headers_t {
    ethernet_t ethernet;
    my_tunnel_t my_tunnel;
    ipv4_t ipv4;
    packet_out_header_t packet_out;
    packet_in_header_t packet_in;
}
```

To implement tunnel in the header, we need tunnel id and protocol ip to identify the tunneling protocol and IPv4 protocol:

```
header my_tunnel_t {
    bit<16> proto_id;
    bit<32> tun_id;
}
```

As we can observe from the header structure in the instruction, if we want to implement a IPv4 header, we need components of `version`, `ihl`, `diffsev`, `len`, `identification`, `flags`, `flag offset`, `ttl`, `protocol`, `header checksum`, `src address` and `dst address`. The number of bits of each component can be inferred from the header table provided by the instructions. So we can implement the `ipv4_t` as follows:

```
header ipv4_t {
    bit<4>  version;
    bit<4>  ihl;
    bit<8>  diffserv;
    bit<16> len;
    bit<16> identification;
    bit<3>  flags;
    bit<13> frag_offset;
    bit<8>  ttl;
    bit<8>  protocol;
    bit<16> hdr_checksum;
    bit<32> src_addr;
    bit<32> dst_addr;
}
```

By now we have successfully implemented the header structure.

2.1.2 Parser Implementation

After some document reading, we know that a P4 parser is a state machine, it contains several states. The initial state is `start`, and the final state is `accept`. Each intermediate state can specify the next state by using a select statement over the header fields extracted. So at first let us implement the `start` state. In this state, we need to parse the CPU port in the packet header. So in the top of the file, we can construct a `CPU_port` like this:

```
const port_t CPU_PORT = 255;
```

After that, we need to define another state to parse the CPU port, which is `parse_packet_out`, we can assign this state to the `CPU_port` in the `start` state. Then, we can go to the next step, which is `parse_ethernet`, by set `parse_ethernet` as the default next state in the `start` state. So the `start` state will be like this:

```
    state start {
        transition select(standard_metadata.ingress_port) {
            CPU_PORT: parse_packet_out;
            default: parse_ethernet;
        }
    }
```

In the `parse_packet_out` state, we need to parse the `packet_out` attribution in the packet header, and assign it to the `CPU_port` attribution. Then transit to the next state.

```
    state parse_packet_out {
        packet.extract(hdr.packet_out);
        transition parse_ethernet;
    }
```

In the `parse_ethernet` state, we need to parse `my_tunnel` and `ipv4` attributions in the packet header. In this case, we need to define the attribution that we want the parsed values to be assigned to, which are `ETH_TYPE_MYTUNNEL` and `ETH_TYPE_IPV4`. We can define them on the top of the file like this(`ETH_TYPE_IPV4` is already defined):

```
    const bit<16> ETH_TYPE_MYTUNNEL = 0x1212;
    const bit<16> ETH_TYPE_IPV4 = 0x800;
```

To parse these two attributions, we can use another two states `parse_my_tunnel` and `parse_ipv4` to accomplish that. So by now we can implement the `parse_ethernet` state as follows:

```
    state parse_ethernet {
        packet.extract(hdr.ethernet);
        transition select(hdr.ethernet.ether_type) {
            ETH_TYPE_MYTUNNEL: parse_my_tunnel;
            ETH_TYPE_IPV4: parse_ipv4;
            default: accept;
        }
    }
```

To parse `my_tunnel` attribution in the packet header, we can use the state as follows:

```
state parse_my_tunnel {
    packet.extract(hdr.my_tunnel);
    transition select(hdr.my_tunnel.proto_id) {
        ETH_TYPE_IPV4: parse_ipv4;
        default: accept;
    }
}
```

To parse `ipv4` attribution in the packet header, we can use the state as follows:

```
state parse_ipv4 {
    packet.extract(hdr.ipv4);
    transition accept;
}
```

By now we can say that the parser has been implemented. However, accordingly, we should not forget to implement `my_tunnel` and `ipv4` deparsing in the `c_deparser` function of this file:

```
packet.emit(hdr.my_tunnel);
packet.emit(hdr.ipv4);
```

2.1.3 Match and Action Tables Implementation

In `c_ingress` function, we need to implement three match and action tables, which are layer 2 forwarding table, tunnel ingress table and tunnel forwarding table.

In layer 2(AKA l2) forwarding, on receiving a frame via. an input port, after updating the ARL table, the next task of the L2 Switch is to forward the frame onto the correct output port on which the frames destination end node is present.

To find this out, the L2 Switch does a ARL table lookup to find a matching entry for the destination MAC address of the incoming frame. If a match is found, then the frame is forwarded only to that port, provided that the destination port is different than the one on which the frame arrived. If the destination port is the same port through which the frame was received, then the L2 Switch does not do anything, as it knows that the destination end node would have already received this frame.

From this, we can see that a l2 forwarding table needs keys of ingress port, destination port, source port and ethernet type, and actions of setting packet out port, sending packet to CPU and dropping packet, which will be implemented later.

Also, we need a l2 forwarding counter to count packets and bytes matched by each entry of the table:

```
direct_counter(CounterType.packets_and_bytes) l2_fwd_counter;
```

With the information provided above, we can implement the l2 forwarding table as follows, it provides basic L2 forwarding capabilities and actions to send packets to the controller:

```
table t_l2_fwd {
    key = {
        standard_metadata.ingress_port  : ternary;
        hdr.ethernet.dst_addr           : ternary;
        hdr.ethernet.src_addr           : ternary;
        hdr.ethernet.ether_type         : ternary;
    }
    actions = {
        set_out_port;
        send_to_cpu;
        _drop;
        NoAction;
    }
    default_action = NoAction();
    counters = l2_fwd_counter;
}
```

In the tunnel ingress table, we need the key of Longest Prefix Match(lpm) matches with the longest subnet mask to enter the tunnel. Also we need the action of tunnel ingress(`my_tunnel_ingress`) to enter the tunnel, which will be implemented later. So we can implement the tunnel ingress table as follows:

```
table t_tunnel_ingress {
    key = { hdr.ipv4.dst_addr: lpm; }
    actions = {
        my_tunnel_ingress;
        _drop;
    }
    default_action = _drop();
}
```

In the tunnel forwarding table, we need the field value which is exactly the same as the table value(`Exact`). Also we need the action of tunnel egress(`my_tunnel_egress`) to exit the tunnel after forwarding, which will be implemented later. So we can implement the tunnel forwarding table as follows:

```
table t_tunnel_fwd {
    key = {
        hdr.my_tunnel.tun_id: exact;
    }
    actions = {
        set_out_port;
        my_tunnel_egress;
        _drop;
    }
    default_action = _drop();
}
```

After implementing the tables, we can work on implementing the actions inside the tables. At first, in the `t_12_fwd` table, there are three actions, `send_to_cpu`, `set_out_port` and `_drop`.

For `send_to_cpu`, we need to set the egress port to the `CPU_port`, and set the ingress port properly with the standard metadata. Also, we need to set `packet_in` attribution in the header as valid, because packets sent to the controller needs to be prepended with the packet-in header. By setting it valid we make sure it will be deparsed on the wire. So we can implement this action as follows:

```
action send_to_cpu() {
    standard_metadata.egress_spec = CPU_PORT;
    hdr.packet_in.setValid();
    hdr.packet_in.ingress_port = standard_metadata.ingress_port;
}
```

For action `set_out_port`, we specifies the output port for this packet by setting the corresponding metadata as follows:

```
action set_out_port(port_t port) {
    standard_metadata.egress_spec = port;
}
```

For action _drop, we can simply mark the packet to drop:

```
action _drop() {
    mark_to_drop();
}
```

As is shown in the t_tunnel_ingress table, there would be an action called my_tunnel_ingress. We utilize this action to set the tunnel valid, select the right tunnel by assigning tunnel id, and set the ethernet type and protocol.

```
action my_tunnel_ingress(bit<32> tun_id) {
    hdr.my_tunnel.setValid();
    hdr.my_tunnel.tun_id = tun_id;
    hdr.my_tunnel.proto_id = hdr.ethernet.ether_type;
    hdr.ethernet.ether_type = ETH_TYPE_MYTUNNEL;
}
```

As we can see from the t_tunnel_egress table, there would be an action called my_tunnel_egress, we use it to set the egress port, change ethernet type to original protocol id, and finally, of course, set the tunnel to invalid.

```
action my_tunnel_egress(bit<9> port) {
    standard_metadata.egress_spec = port;
    hdr.ethernet.ether_type = hdr.my_tunnel.proto_id;
    hdr.my_tunnel.setInvalid();
}
```

2.1.4 Control Implementation

Now it is the time to design the control flow. We can apply this control function to every packet received by this switch.

At first, we should judge if the packet is from the CPU_port. If it is, this is a packet-out sent by the controller. So we just skip table processing and set the egress port as requested by the controller (packet_out header) and remove the packet_out header(set it as invalid).

```
if (standard_metadata.ingress_port == CPU_PORT) {
    standard_metadata.egress_spec = hdr.packet_out.egress_port;
    hdr.packet_out.setInvalid();
}
```

Else, if the packet is not from the `CPU_port`, which means the packet is received from the data plane port, we should apply table `t_l2_fwd` to the packet. If packet hit an entry in `t_l2_fwd` table, a forwarding action has already been taken. So there is no need to apply other tables, we can just then exit this control block.

Then, if while the IPv4 is valid but the tunnel is invalid, we just process only non-tunneled IPv4 packets. If the tunnel is valid, we process all tunneled packets.

So this complete if-else loop would be like this:

```
if (standard_metadata.ingress_port == CPU_PORT) {
    standard_metadata.egress_spec = hdr.packet_out.egress_port;
    hdr.packet_out.setInvalid();
} else {
    if (t_l2_fwd.apply().hit) {
        return;
    }

    if (hdr.ipv4.isValid() && !hdr.my_tunnel.isValid()) {
        t_tunnel_ingress.apply();
    }

    if (hdr.my_tunnel.isValid()) {
        t_tunnel_fwd.apply();
    }
}
```

Finally, the ingress port and egress should be no larger than the `MAX_PORTS`, which can be defined on the top of the file:

```
#define MAX_PORTS 255
```

Also, we need two counters to count packets/bytes received/sent on each port. For each counter we instantiate a number of cells equal to `MAX_PORTS`.

```
counter(MAX_PORTS, CounterType.packets_and_bytes)
    tx_port_counter;
counter(MAX_PORTS, CounterType.packets_and_bytes)
    rx_port_counter;
```

With these, we can update port counters at index = ingress or egress port after the previous if-else loop.

```
    if (standard_metadata.egress_spec < MAX_PORTS) {
        tx_port_counter.count((bit<32>)
            standard_metadata.egress_spec);
    }
    if (standard_metadata.ingress_port < MAX_PORTS) {
        rx_port_counter.count((bit<32>)
            standard_metadata.ingress_port);
    }
```

So this whole control flow will be like this:

```
    apply {
        if (standard_metadata.ingress_port == CPU_PORT) {
            standard_metadata.egress_spec =
                hdr.packet_out.egress_port;
            hdr.packet_out.setInvalid();
        } else {
            if (t_l2_fwd.apply().hit) return;
            if (hdr.ipv4.isValid() && !hdr.my_tunnel.isValid()) {
                t_tunnel_ingress.apply();
            }
            if (hdr.my_tunnel.isValid()) t_tunnel_fwd.apply();
        }

        if (standard_metadata.egress_spec < MAX_PORTS) {
            tx_port_counter.count((bit<32>)
                standard_metadata.egress_spec);
        }
        if (standard_metadata.ingress_port < MAX_PORTS) {
            rx_port_counter.count((bit<32>)
                standard_metadata.ingress_port);
        }
    }
```

2.2 MyTunnelApp Implementation

2.2.1 Insert Flow Rules

At first, we can retrieve tunnel ingress table ID from the given tunnel ID by:

```
PiTableId tunnelIngressTableId =
    PiTableId.of("c_ingress.t_tunnel_ingress");
```

Next, we can get the longest prefix match on IPv4 destination address by:

```
PiMatchFieldId ipDestMatchFieldId =
    PiMatchFieldId.of("hdr.ipv4.dst_addr");
PiCriterion match = PiCriterion.builder()
        .matchLpm(ipDestMatchFieldId, dstIpAddr.toOctets(), 32)
        .build();
```

Then, we can build the action to add to the ingress table by:

```
PiActionParam tunIdParam =
    new PiActionParam(PiActionParamId.of("tun_id"), tunId);
PiActionId ingressActionId =
    PiActionId.of("c_ingress.my_tunnel_ingress");
PiAction action = PiAction.builder()
        .withId(ingressActionId)
        .withParameter(tunIdParam)
        .build();
```

To better maintain the program, we can add logs of the ingress rule information by:

```
log.info("Inserting INGRESS rule on switch {}:
    table={}, match={}, action={}",
    switchId, tunnelIngressTableId, match, action);
```

Finally, we can insert the flow rules to finish this subtask.

```
insertPiFlowRule(switchId, tunnelIngressTableId, match, action);
```

2.2.2 Insert Flow Rules

Now we are implementing the action which depends on the isEgress parameter. If it is true, perform tunnel egress action on the given outPort, otherwise simply forward packet as is (`set_out_port` action).

Here is the implement of `true` case:

```
PiActionId egressActionId =
    PiActionId.of("c_ingress.my_tunnel_egress");
    action = PiAction.builder()
            .withId(egressActionId)
            .withParameter(portParam)
            .build();
```

Similarly, for the transit case, we can check the `t_tunnel_fwd` table in the p4 program to get the action name to set the output port, which is `set_out_port`. It can be used to create the `PiActionId` object.

```
PiActionId egressActionId =
    PiActionId.of("c_ingress.set_out_port");
    action = PiAction.builder()
        .withId(egressActionId)
        .withParameter(portParam)
        .build();
```

2.2.3 Insert PI Flow Rules

For systems that use PI criterion and action, we can implement rules like this:

```
FlowRule rule = DefaultFlowRule.builder()
    .forDevice(switchId)
    .forTable(tableId)
    .fromApp(appId)
    .withPriority(FLOW_RULE_PRIORITY)
    .makePermanent()
    .withSelector(DefaultTrafficSelector.builder()
    .matchPi(piCriterion).build())
    .withTreatment(DefaultTrafficTreatment.builder()
    .piTableAction(piAction).build())
    .build();
```

2.3 Testing Commands

After running `pingall` command on `mininet`, we can see the reachability of the network: